DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Closure

CMSC 124 Activity 14

Prepared by: Rene Andre Bedonia Jocsing

Published: October 29, 2026 **Deadline:** October 31, 2026

Calling a returned function raises a question: where do its surrounding variables live after the call that created it ends? You'll trace a function together with the values it keeps.

You have 60 minutes for the three checkpoints. Keep each worked example beside the checkpoint it supports. Discussion with classmates and the instructor is allowed and encouraged throughout. This activity is worth 16 points. Nine points reward constructions, and seven reward explanations.

The examples use C++14 because its syntax makes value capture and reference capture visible. You aren't expected to know C++ beyond this sheet. Read `int` as an integer type and `double` as a floating-point type. Here, `3` has type `int`, while `1.5` and `2.0` have type `double`.

Each worked example and checkpoint repeats the definitions it needs. Treat each as a separate program. Read the statements inside `main`, the function where a C++ program starts, in order. You only need to trace them on paper.

Overloads and a Generic

These two functions share an identifier but accept different parameter types. Each is an **overload** of `scale`:

```
int scale(int value, int factor) {
    return value * factor;
}

double scale(double value, double factor) {
    return value * factor;
}

template<class T>
T twice(T value) {
    return value + value;
}
```

`twice` is a **generic function**: one definition describes a family of functions with different types. Read `template<class T>` as "a template with type parameter T." **Template deduction** determines `T` from the argument's type. For example, `twice(5)` uses `T = int`, so its parameter and result both have type `int`.

The compiler selects the overload and template instance from types, before the program runs. In these examples, both arguments exactly match one overload's parameter types. Trace the inner call first. Its result type supplies the argument type for the outer call.

Worked Example for Checkpoint 1

Suggested time: 15 minutes, including the worked example.

Trace these addition calls:

```
int add(int left, int right) {
    return left + right;
}

double add(double left, double right) {
    return left + right;
}

template<class T>
T twice(T value) {
    return value + value;
}

int main() {
    twice(add(2, 3));
    twice(add(1.5, 2.0));
}
```

Step	`twice(add(2, 3))`	`twice(add(1.5, 2.0))`
Read the inner argument types	`int`, `int`	`double`, `double`
Select the matching overload	`add(int, int)`	`add(double, double)`
Compute the inner result	`int` 5	`double` 3.5
Deduce `T` for `twice`	`T = int`, use `twice<int>`	`T = double`, use `twice<double>`
Compute the final result	`5 + 5`, giving `int` 10	`3.5 + 3.5`, giving `double` 7.0

The type travels outward. When the selected functions run, they compute the values along these paths:

Resolve the inner call before the outer call

Integer arguments

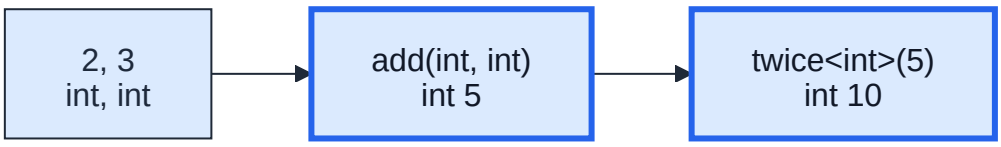


Figure 1: Integer arguments

Double arguments

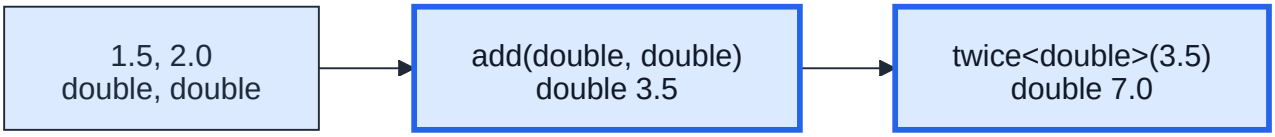


Figure 2: Double arguments

Checkpoint 1: Choose the Callable (4 Points)

Trace these multiplication calls:

```cpp
int scale(int value, int factor) {
    return value * factor;
}

double scale(double value, double factor) {
    return value * factor;
}

template<class T>
T twice(T value) {
    return value + value;
}

int main() {
    twice(scale(3, 4));
    twice(scale(1.5, 2.0));
}
```

1. (Construction: 2 points) Make one trace per expression. Include the inner argument types, selected `scale` overload, intermediate type and value, `T`, and final type and value. Each complete trace is worth 1 point.

2. (Explanation: 2 points) Explain which argument types select each `scale` overload (1 point). Then explain how the inner result's type determines `T` (1 point).

Return a Function

A **first-class function** can be stored, passed, or returned as a value. A C++ **lambda** is an anonymous function expression that creates a callable object. You'll return that object and store it in a variable.

A **factory** is a function that creates and returns a value. This factory returns a lambda:

```cpp
int scale(int value, int factor) {
    return value * factor;
}

int factor = 100;                  // global variable

auto make_multiplier(int factor) {
    return [factor](int value) {
        return scale(value, factor);
    };
}
```

Read `[factor](int value) { ... }` in order:

1. `[factor]` is the **capture list**. It copies the surrounding parameter's value when the lambda is created.
2. `(int value)` is the lambda's parameter list. It receives an integer when someone calls the lambda.
3. `{ ... }` is the body that runs on that later call. Creating the lambda doesn't run its body.

`auto` in the factory's return position asks the compiler to deduce its return type from the returned object. In `auto by2 = make_multiplier(2)`, it deduces the variable's type from the initializer. Neither use means that the variable has no type.

A **closure** combines callable code with a captured **environment**, the values or references it keeps for later calls. The factory returns this callable object, not the result of `scale`. In `auto by2 =`

`make_multiplier(2)`, `by2` therefore holds a closure. Only a later call such as `by2(7)` runs its body and returns an integer.

Draw the closure as `<code, environment>`. To build the drawing, separate the body from the capture list:

Part	What to put in the drawing for `make_multiplier(2)`
Code	The body `return scale(value, factor);`
Environment	An owned copy labeled `factor = 2`, created by `[factor]`
Link	Connect that code to that environment. Together they describe the returned closure.

Don't put `value` in the captured environment. It is the lambda's parameter, supplied afresh on each later call. For `by2(7)`, the body receives `value = 7` and reads its captured `factor = 2`. The function `scale` already exists outside the factory. It isn't a captured variable.

Lexical binding resolves an identifier using the scopes around its definition. The factory parameter `factor` **shadows** the global: inside the factory, that name refers to the parameter. `[factor]` therefore copies the parameter's 2, not global 100. Calling the closure later doesn't select a different `factor` from the caller's surroundings.

Each factory call creates a new closure with its own copy. Calling `make_multiplier(5)` creates another environment containing `factor = 5`. It doesn't replace the first closure's 2. Draw one shared body with a separate link to each environment. The original parameters end when their factory calls return. The captured copies belong to the returned objects.

This drawing is a conceptual model of behavior. It doesn't specify a C++ object layout, require a stored code pointer, or imply allocation on the heap.

Worked Example for Checkpoint 2

Suggested time: 30 minutes, including the worked example.

Create a closure, then call it after the factory returns:

```
int add(int left, int right) {
    return left + right;
}

double add(double left, double right) {
    return left + right;
}

template<class T>
T twice(T value) {
    return value + value;
}

int amount = 100;

auto make_offset(int amount) {
    return [amount](int value) {
        return add(value, amount);
    };
}

int main() {
```

```
23      auto plus2 = make_offset(2);
24      int total = twice(plus2(3));
25  }
```

A **cell** is a storage location that holds a value. Follow the original parameter cell and the closure's copy separately:

Step	Original factory parameter	Closure and call
Enter `make_offset(2)`	Create `amount = 2`	No closure yet
Evaluate `[amount](int value) { ... }`	Parameter still holds 2	Create the closure with its own copy 2. Don't run `add` yet.
Return and initialize `plus2`	Parameter lifetime ends	`plus2` retains the code and copied 2
Call `plus2(3)`	Original parameter no longer exists	`value` is 3. `add(int, int)` reads 3 and copied 2, giving `int` 5.
Call the outer `twice`	Original parameter still absent	Deduce `T = int`. `twice<int>(5)` returns 10, stored in `total`.

The global `amount` remains 100. Lexical binding selected the factory parameter, and value capture kept a copy of 2 for the later call.

A copy survives the factory call

Create and return the closure

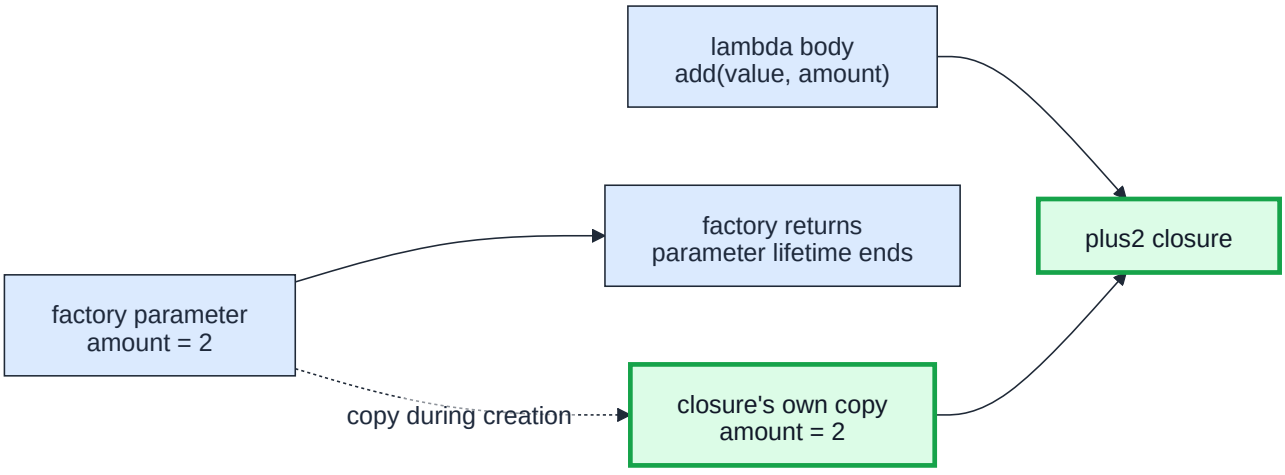


Figure 3: Create and return the closure

Invoke after the factory returns

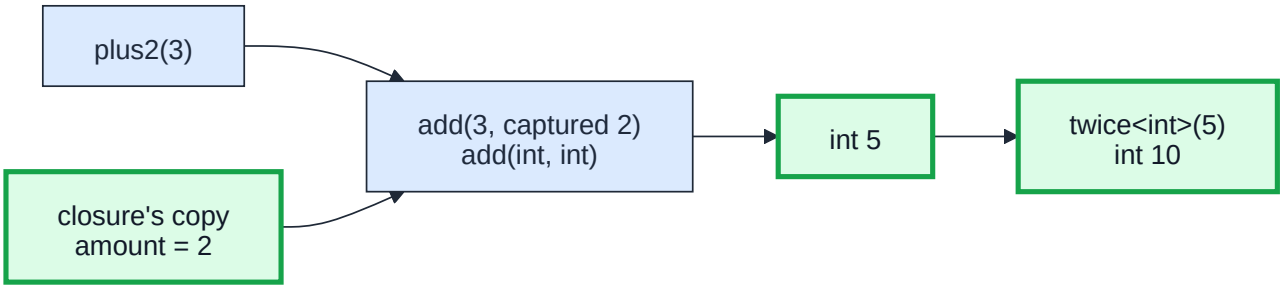


Figure 4: Invoke after the factory returns

The original parameter's lifetime didn't extend. The copy remains available while the closure exists. **Binding extent** describes how long a binding remains available, so distinguish the original parameter's extent from access to the closure's copy.

Laboratory Activity 5 uses the same code-plus-environment model when you implement function values in your own language.

Checkpoint 2: Draw Code Plus Environment (7 Points)

Trace this program:

```
int scale(int value, int factor) {
    return value * factor;
}

double scale(double value, double factor) {
    return value * factor;
}

template<class T>
T twice(T value) {
    return value + value;
}

int factor = 100;

auto make_multiplier(int factor) {
    return [factor](int value) {
        return scale(value, factor);
    };
}

int main() {
    auto by2 = make_multiplier(2);
    auto by3 = make_multiplier(3);

    int a = twice(by2(7));
    int b = by3(7);
}
```

1. (Construction: 4 points) Draw one shared lambda body linked to two separate environments, one for `by2` and one for `by3` (1 point each). Label each captured value. Then trace the initializations of `a` and `b` (1 point each). Include the selected `scale` overload, each intermediate type and value, and any `twice` instance. State explicitly when a trace has no `twice` call.

2. (Explanation: 3 points) Explain why neither closure uses global `factor = 100` (1 point). Explain why the two factory calls create separate captured copies (1 point). Explain what ends at factory return and what remains available for later calls (1 point).

Transfer the Capture Method

The **capture method** determines whether the closure keeps a value or refers to an existing cell. `[factor]` captures by value. `[&factor]` captures by reference, so the lambda accesses the original parameter cell. Here, `&` belongs to the capture syntax. You aren't writing a pointer expression.

Worked Example for Checkpoint 3

Suggested time: 15 minutes, including the worked example.

Here, reference capture retains access to the factory parameter:

```
int add(int left, int right) {
    return left + right;
}

auto make_bad_offset(int amount) {
    return [&amount](int value) {
        return add(value, amount);
    };
}

int main() {
    auto bad_offset = make_bad_offset(2);
    int unsafe_total = bad_offset(3);
}
```

Step	Parameter cell	Returned closure
Enter the factory	Create `amount = 2`	No closure yet
Create the lambda	Parameter still exists	Retain a reference to that cell
Return from the factory	Parameter lifetime ends	The reference now dangles
Call `bad_offset(3)`	Parameter no longer exists	The body tries to read through the dangling reference

A **dangling reference** refers to an object whose lifetime ended. Reading through it here causes **undefined behavior**. C++ specifies no required outcome. Even if one run produces 5, `unsafe_total` has no language-defined value. This is a defect in the program.

Repair the factory by replacing `[&amount]` with `[amount]`:

```
int add(int left, int right) {
    return left + right;
}

auto make_safe_offset(int amount) {
    return [amount](int value) {
        return add(value, amount);
    };
}

int main() {
    auto safe_offset = make_safe_offset(2);
```

```
    int safe_total = safe_offset(3);    // 5
}
```

The repaired closure owns a copy of 2. The original parameter still ends at return, but the later addition reads the copy. Remembering the old number doesn't restore an expired object's lifetime.

Reference capture can be valid. If a lambda refers to a caller's cell, that cell must still exist whenever the lambda accesses it. Returning a closure doesn't automatically extend the referenced object's lifetime.

Checkpoint 3: Refuse the Fake Result (5 Points)

Trace this program:

```
int scale(int value, int factor) {
    return value * factor;
}

auto make_bad_multiplier(int factor) {
    return [&factor](int value) {
        return scale(value, factor);
    };
}

int main() {
    auto bad = make_bad_multiplier(4);
    int result = bad(5);
}
```

1. (Construction: 3 points) Draw the parameter's lifetime and the returned closure's reference. Mark when the reference dangles (1 point). State whether the original `result` has a language-defined value (1 point). Rewrite only the capture list. Give the repaired call's type and value (1 point).

2. (Explanation: 2 points) Explain which object ends at factory return (1 point). Explain why its former value 4 doesn't make the later reference access valid (1 point).